

TRƯỜNG ĐẠI HỌC CÔNG NGHỆ THÔNG TIN, ĐHQG-HCM
KHOA KỸ THUẬT MÁY TÍNH



BÁO CÁO THỰC HÀNH LAB3
MÔN THIẾT KẾ HỆ THỐNG SỐ VỚI HDL

Giảng viên hướng dẫn: Ngô Hiếu Trường

Thực hiện bởi sinh viên:

1. Nguyễn Hoàng Quốc Cường - 23520200

MỤC LỤC

MỤC LỤC	2
DIRECT MEMORY ACCESS	3
Chương I. TỔNG QUAN.	3
1. Giới thiệu.....	3
2. Cơ sở lý thuyết.....	3
2.1 Kiến trúc hệ thống DMA	3
2.2 Mô tả các khối chức năng	4
Chương II. HIỆN THỰC VÀ GIẢI QUYẾT VẤN ĐỀ	7
1. Thực hiện mô phỏng.....	7
2. Xử lý file lỗi (FIFO.v)	10
Chương III. KẾT LUẬN.	14
1. Đánh giá định tính	Error! Bookmark not defined.
2. Đánh giá định lượng	Error! Bookmark not defined.
3. Kết luận về giải thuật.....	Error! Bookmark not defined.

DIRECT MEMORY ACCESS

CHƯƠNG I. TỔNG QUAN.

1. Giới thiệu

Trong các hệ thống vi xử lý, việc truyền dữ liệu khối lượng lớn giữa bộ nhớ và ngoại vi (hoặc giữa các vùng nhớ) nếu thực hiện hoàn toàn bởi CPU sẽ gây lãng phí tài nguyên xử lý. CPU phải tốn nhiều chu kỳ lệnh cho việc đọc/ghi và kiểm tra trạng thái, dẫn đến hiệu suất hệ thống giảm.

Để giải quyết vấn đề này, cơ chế DMA (Direct Memory Access) được sử dụng. DMA cho phép truyền dữ liệu trực tiếp giữa bộ nhớ và thiết bị ngoại vi mà không cần sự can thiệp liên tục của CPU, giúp giải phóng CPU để thực hiện các tác vụ khác.

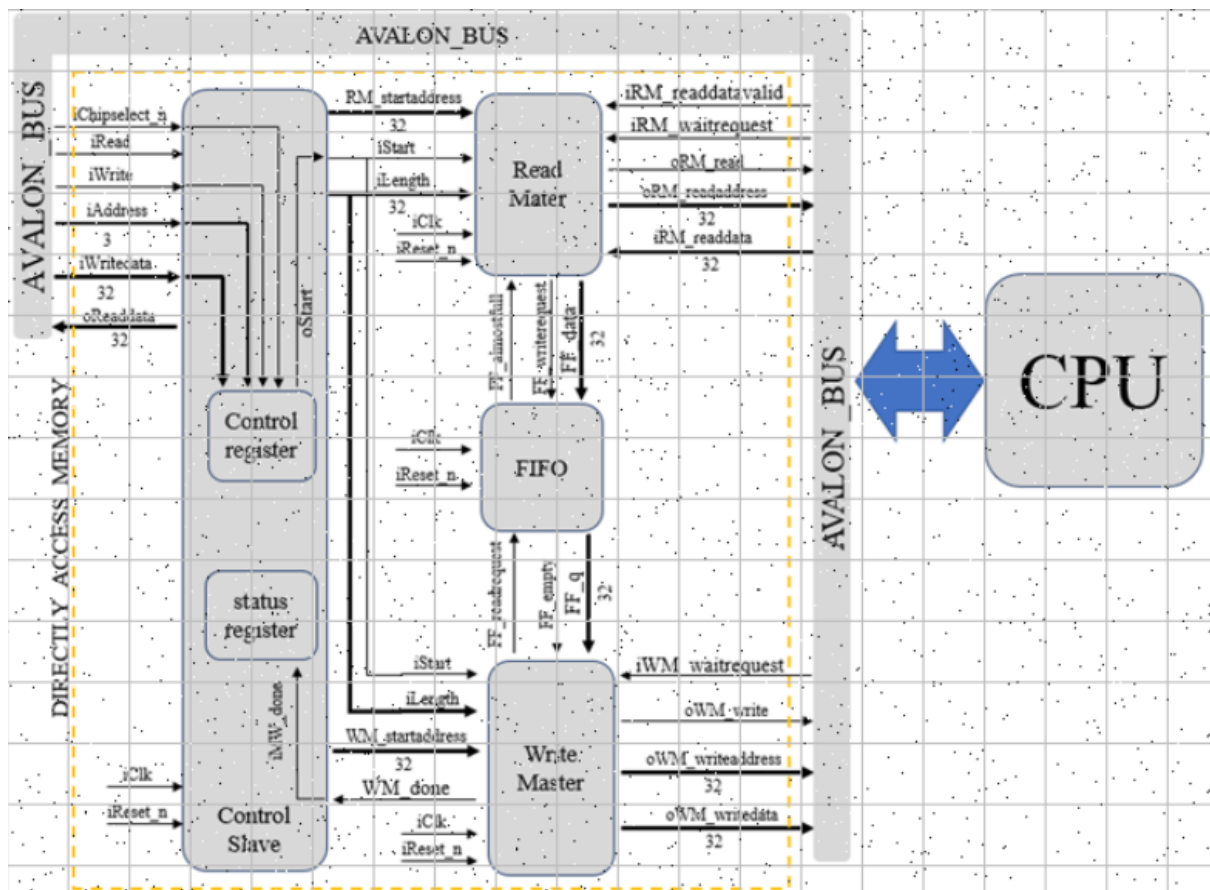
2. Cơ sở lý thuyết.

2.1 Kiến trúc hệ thống DMA

Hệ thống được thiết kế dựa trên kiến trúc Avalon Interface của Altera (Intel FPGA). Bộ DMA đóng vai trò trung gian nhận lệnh từ CPU và thực hiện sao chép dữ liệu.

Sơ đồ khối hệ thống bao gồm các module sau:

- **Control Slave**
- **Read Master**
- **Write Master**
- **FIFO**



Hình 1: Sơ đồ khối hệ thống DMA

2.2 Mô tả các khối chức năng

2.2.1 Khối Control Slave

Module này đóng vai trò Slave Interface, cho phép CPU Nios II cấu hình và điều khiển hoạt động của DMA.

Chức năng: Nhận các lệnh từ CPU thông qua tín hiệu *iChipselect_n*, *iWrite* và lưu trữ vào các thanh ghi nội bộ:

- Địa chỉ nguồn/ *RM_startaddress*
- Địa chỉ đích/ *WM_startaddress*
- Độ dài dữ liệu/ *Length*
- Lệnh bắt đầu/ *Start*

Giao tiếp: Khi quá trình DMA hoàn tất (nhận tín hiệu *iMW_done*), nó sẽ tự động xóa bit *Start* để sẵn sàng cho lần truyền tiếp theo.

2.2.2 Khối Read Master

Module này đóng vai trò Master Interface chịu trách nhiệm đọc dữ liệu từ bộ nhớ nguồn.

- **Hoạt động:** Khi nhận tín hiệu Start, khối này phát sinh địa chỉ đọc *oRM_readaddress* và tín hiệu đọc *oRM_read* lên bus Avalon.
- **Luồng dữ liệu:** Dữ liệu đọc được từ bộ nhớ (*iRM_readdata*) sẽ được đẩy vào FIFO thông qua tín hiệu *oFF_writerequest* khi tín hiệu *iRM_readdatavalid* tích cực. Nó sẽ tạm dừng đọc nếu FIFO báo đầy (*iFF_almostfull*).

2.2.3 Khởi Write Master

Module này chịu trách nhiệm lấy dữ liệu từ bộ đệm và ghi sang bộ nhớ đích.

- **Hoạt động:** Kiểm tra trạng thái FIFO, nếu FIFO không rỗng (*iFF_empty* = 0), nó sẽ đọc dữ liệu từ FIFO và phát sinh chu kỳ ghi (*oWM_write*) xuống địa chỉ đích (*oWM_writeaddress*).
- **Kết thúc:** Khi ghi đủ số lượng byte yêu cầu (*iLength*), nó phát sinh tín hiệu *oWM_done* để báo cáo hoàn thành.

2.2.4 Khởi FIFO

Bộ nhớ đệm trung gian (First-In-First-Out) giúp đồng bộ hóa tốc độ giữa quá trình đọc và ghi, đảm bảo dữ liệu không bị mất mát.

- Sử dụng RAM phân tán hoặc thanh ghi để lưu trữ dữ liệu.
- Cung cấp các cờ trạng thái *FF_empty* và *FF_almostfull* để điều phối hoạt động của Read Master và Write Master.

2.2.5 Module kết nối

Module DMAFinal có nhiệm vụ kết nối các module thành phần với nhau, đảm bảo truyền/ nhận dữ liệu, đảm bảo thống nhất các module con

2.3 Mô tả các tín hiệu theo nhóm (dựa trên Qsys)

2.3.1 Nhóm tín hiệu toàn cục

iClk: Xung clock của toàn hệ thống

iReset_n: Tín hiệu reset tích cực cạnh xuống

2.3.2 Nhóm tín hiệu điều khiển(Avalon-MM Slave Interface)

iChipselect_n: Tín hiệu chọn chip (Active Low). Khi bằng 0, module Control Slave được kích hoạt để giao tiếp với CPU.

iRead: Cờ báo CPU muốn đọc dữ liệu từ thanh ghi DMA.

iWrite: Cờ báo CPU muốn ghi dữ liệu cấu hình vào thanh ghi DMA.

iAddress: Địa chỉ thanh ghi (3-bit) mà CPU muốn truy xuất (Reg0 - Reg7).

iWritedata: Dữ liệu 32-bit từ CPU gửi xuống để ghi vào thanh ghi.

oReaddata: Dữ liệu 32-bit từ DMA gửi trả về cho CPU.

2.3.3 Nhóm tín hiệu Đọc dữ liệu (Avalon-MM Read Master Interface)

oRM_read: Lệnh đọc. Khi tích cực (mức 1), DMA yêu cầu bộ nhớ nguồn cung cấp dữ liệu.

oRM_readaddress: Địa chỉ bộ nhớ nguồn mà DMA đang muốn đọc.

iRM_readdata: Dữ liệu thực tế được bộ nhớ nguồn trả về cho DMA.

iRM_readdatavalid: Tín hiệu báo dữ liệu hợp lệ. Khi *readdatavalid* = 1, nghĩa là *iRM_readdata* đang chứa dữ liệu đúng, DMA sẽ đẩy dữ liệu này vào FIFO.

iRM_waitrequest: Tín hiệu chờ từ Slave (Bộ nhớ). Nếu bộ nhớ bận, nó sẽ bật tín hiệu này lên 1, yêu cầu DMA giữ nguyên trạng thái và chờ đợi.

2.3.4 Nhóm tín hiệu Ghi dữ liệu (Avalon-MM Write Master Interface)

oWM_write: Lệnh ghi. Khi tích cực, DMA yêu cầu bộ nhớ đích nhận dữ liệu.

oWM_writeaddress: Địa chỉ bộ nhớ đích mà DMA sẽ ghi dữ liệu vào.

oWM_writedata: Dữ liệu được lấy từ FIFO để ghi xuống bộ nhớ đích.

iWM_waitrequest: Tín hiệu chờ từ bộ nhớ đích. Nếu bộ nhớ chưa kịp xử lý, DMA phải dừng quá trình ghi và chờ tín hiệu này về 0.

2.3.5 Nhóm tín hiệu trong FIFO

FF_almostfull: Báo FIFO gần đầy, Read Master sẽ dừng đọc dữ liệu từ nguồn tránh làm tràn bộ đệm

FF_empty: Báo hiệu rỗng, Write Master ngưng ghi ra đích để tránh sai dữ liệu.

FF_writerequest: Khi *readdatavalid* của Read Master tích cực, nó sẽ kích hoạt tín hiệu này để yêu cầu lưu dữ liệu cho vào FIFO

FF_readrequest: Khi Write Master sẵn sàng ghi ra ngoài FIFO và FIFO không rỗng, nó yêu cầu tín hiệu này để lấy dữ liệu tiếp theo từ *FF_q*.

Start: Tín hiệu đánh dấu cho cả 2 Read/Write Master hoạt động

WM_done: Khi Write Master đã ghi đủ số lượng byte (*Length*), nó gửi xung này về Control Slave để xóa bit Start, kết thúc phiên làm việc.

CHƯƠNG II. HIỆN THỰC VÀ GIẢI QUYẾT VẤN ĐỀ

1. Thực hiện mô phỏng

Trong Nios II for Eclipse, ta có một file cần phải mô phỏng là file “hello_world.c” với nội dung như sau:

```
#define DMA (int *) 0x23020
#include "io.h"
#include "stdio.h"
#include "system.h"

int length = 32;
int control = 8;
int readLength = 20;
int enable = 1;

char pdata0[32] = {0, 1, 2, 3, 4, 5, 6, 7,
                  8, 9, 10, 11, 12, 13, 14, 15,
                  16, 17, 18, 19, 20, 21, 22, 23,
                  24, 25, 26, 27, 28, 29, 30, 31};
char *pdata1 = (char*) (ONCHIP_MEMORY2_1_BASE);

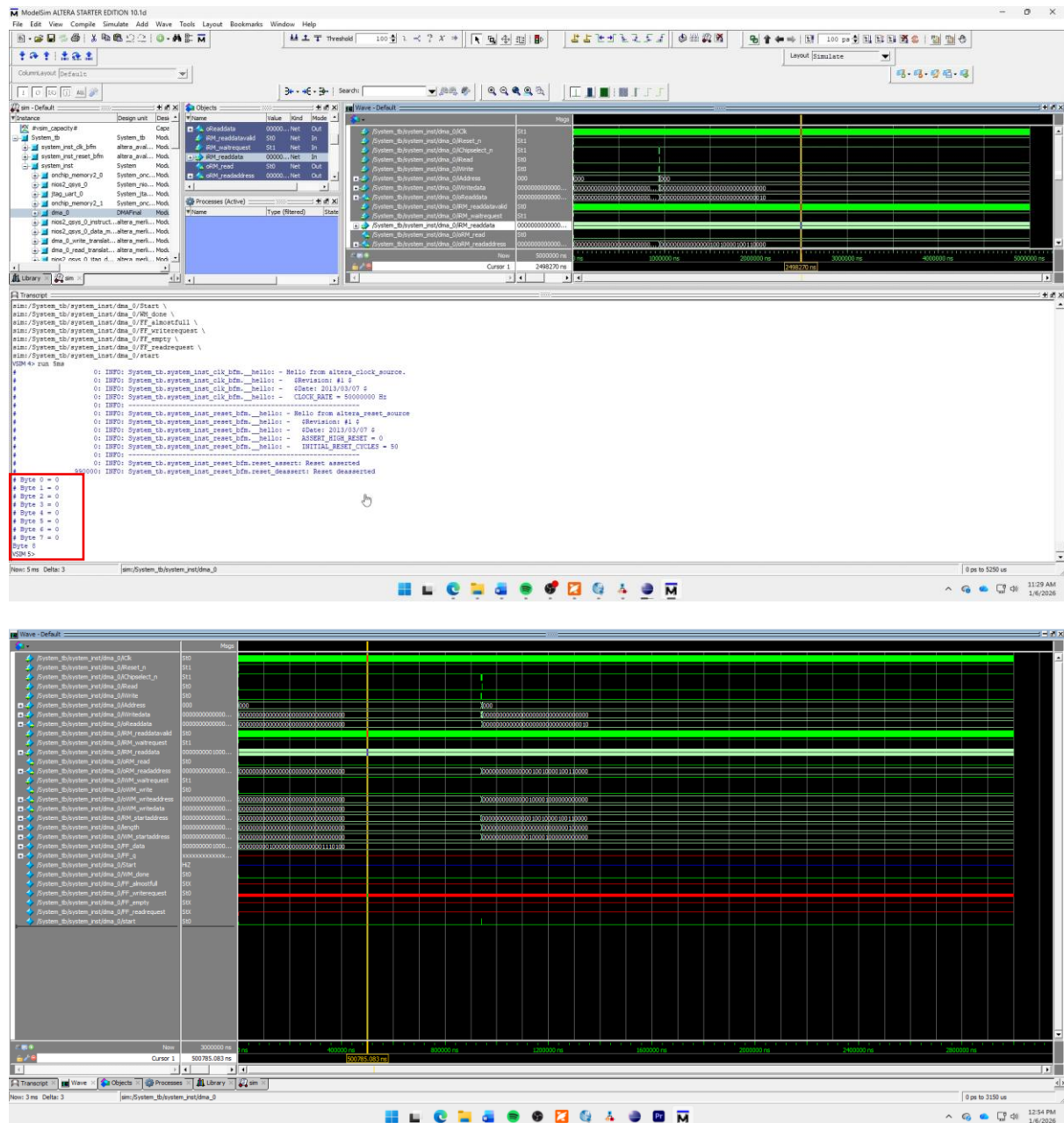
void handlerStatus()
{
    int i;
    for (i = 0; i < 32; i++)
    {
        printf("Byte %d = %d \n", i, pdata1[i]);
    }
}

int main()
{
    char *a = pdata0;
    char *b = pdata1;
    IOWR_32DIRECT(DMA, 0*4, (int)pdata0 );
    IOWR_32DIRECT(DMA, 1*4, (int)pdata1 );
    IOWR_32DIRECT(DMA, 2*4, length);
    IOWR_32DIRECT(DMA, 3*4, control);
    //Xu ly dung polling de kiem tra hoan thanh DMA
    //co do tre 24 chu ky xung clock voi clock dau vao la 50MHz
    //Co the toi uu hon khi tuy chinh lai code C de tao ra iread nhanh hon hoac
    xu dung interrupt
    while(enable)
    {
```

```
        if((IORD_32DIRECT(DMA, 7*4)) <= 2){  
            enable = 0;  
            handlerStatus();  
        }  
    }  
  
    return 0;  
}
```

Bảng 1: Nội dung của file “hello_world.c”

Nội dung chính mà ta cần phải mô phỏng đó chính là khi chạy trên modelsim-altera, ta phải thấy được các con số lần lượt từ 0 tới 31 được in ra tại ô “transcript”. Và cách thực hiện qua video sau: <https://youtu.be/ij4kIFkk5s0>



Hình 2: Kết quả mô phỏng lần đầu

Dựa trên kết quả mô phỏng này, ta có thể thấy được rằng bộ DMA chưa hề chạy khi output ra chỉ toàn số 0, điều này cho thấy dữ liệu từ nguồn không được sao chép thành công sang đích với lý do sau:

1. Trạng thái bất định: Trong môi trường mô phỏng (ModelSim), tại thời điểm khởi chạy tất cả các thanh ghi (reg) chưa được gán giá trị sẽ mặc định ở trạng thái X, Cụ thể ở đây là hai con trỏ quản lý FIFO: pos_read và pos_write đang có giá trị X
2. Lỗi khởi tạo với Reset đồng bộ: Do sử dụng *always @(posedge iClk)*, lệnh Reset (*pos_write <= 0*) chỉ được thực thi nếu tín hiệu iReset_n ở mức thấp **đồng thời** xuất hiện cạnh lên của xung Clock iClk.

- Trong Testbench, nếu tín hiệu Reset được kích hoạt và nhả ra trước khi cạnh lên đầu tiên của Clock xuất hiện, hoặc xung Reset quá ngắn, khối logic bên trong *if(~iReset_n)* sẽ bị bỏ qua.
 - **Hậu quả:** pos_read và pos_write **không được đưa về 0**, chúng tiếp tục giữ trạng thái X.
3. **Quá trình ghi vào FIFO thất bại:** Khi khối **Read Master** đọc dữ liệu từ nguồn và gửi lệnh ghi vào FIFO mà tại thời điểm đọc pos_write là X nên FIFO thực không có dữ liệu gì hoặc chứa dữ liệu rác.
 4. **Quá trình ghi ra RAM đích thất bại:** Tương tự, khi khối **Write Master** cố gắng lấy dữ liệu từ FIFO để ghi ra RAM đích, đọc địa chỉ X từ buffer, dữ liệu nhận được là không xác định, do đó **Write Master** không ghi dữ liệu hợp lệ nào xuống bộ nhớ đích
 5. **Kết quả hiển thị:** Khi chương trình C thực hiện vòng lặp *printf* để đọc giá trị tại RAM đích. Do Write Master không ghi dữ liệu mới vào, CPU thực chất đang đọc **giá trị mặc định ban đầu** của thanh RAM này. Trong hầu hết các trình mô phỏng và cấu hình FPGA, bộ nhớ RAM được khởi tạo mặc định là 0, vậy nên kết quả mới có trường hợp *Byte i = 0* với mọi *i*.

2. Xử lý file lỗi (FIFO.v)

Để chỉnh lại cho mô phỏng có thể in ra được từ 0 tới 31, ta cần phải chỉnh lại file FIFO.v để đổi lại *iReset_n* thành bất đồng bộ.

```
module FIFO(iClk, iReset_n, FF_empty, FF_almostfull, FF_data, FF_q , FF_readrequest,
FF_writerequest);
input iClk, iReset_n, FF_readrequest, FF_writerequest;
output FF_empty, FF_almostfull;
input [31:0] FF_data;
output [31:0] FF_q;

reg [31:0] buffer [256:0]; // bo nho dem cho fifo
reg [7:0] pos_read, pos_write; //dia chi cho doc va ghi cua master_write va master_read
wire fifo_rd, fifo_wr;
wire compare, equal; //2 bits trạng thái dung cho bao FF_almostfull va FF_empty
assign FF_q = buffer[pos_read]; //FF_q luôn được xuất từ fifo
assign fifo_wr = (~FF_almostfull) & FF_writerequest; //Kiểm tra tín hiệu FF_writerequest và
FF_almostfull để tiến hành ghi
assign fifo_rd = (~FF_empty & FF_readrequest); //Kiểm tra tín hiệu FF_readrequest và FF_empty
để tiến hành đọc
//--Ghi dữ liệu vào bộ đếm FIFO

always @ (posedge iClk or negedge iReset_n) //đổi iReset_n thành bất đồng bộ
begin
    if(~iReset_n)
        pos_write <= 8'b00000000;
    else if(fifo_wr)
        begin
            buffer[pos_write] <= FF_data;
```

```

        pos_write <= pos_write + 1;
    end
    // Không cần else pos_write <= pos_write vì Verilog tự giữ giá trị
end

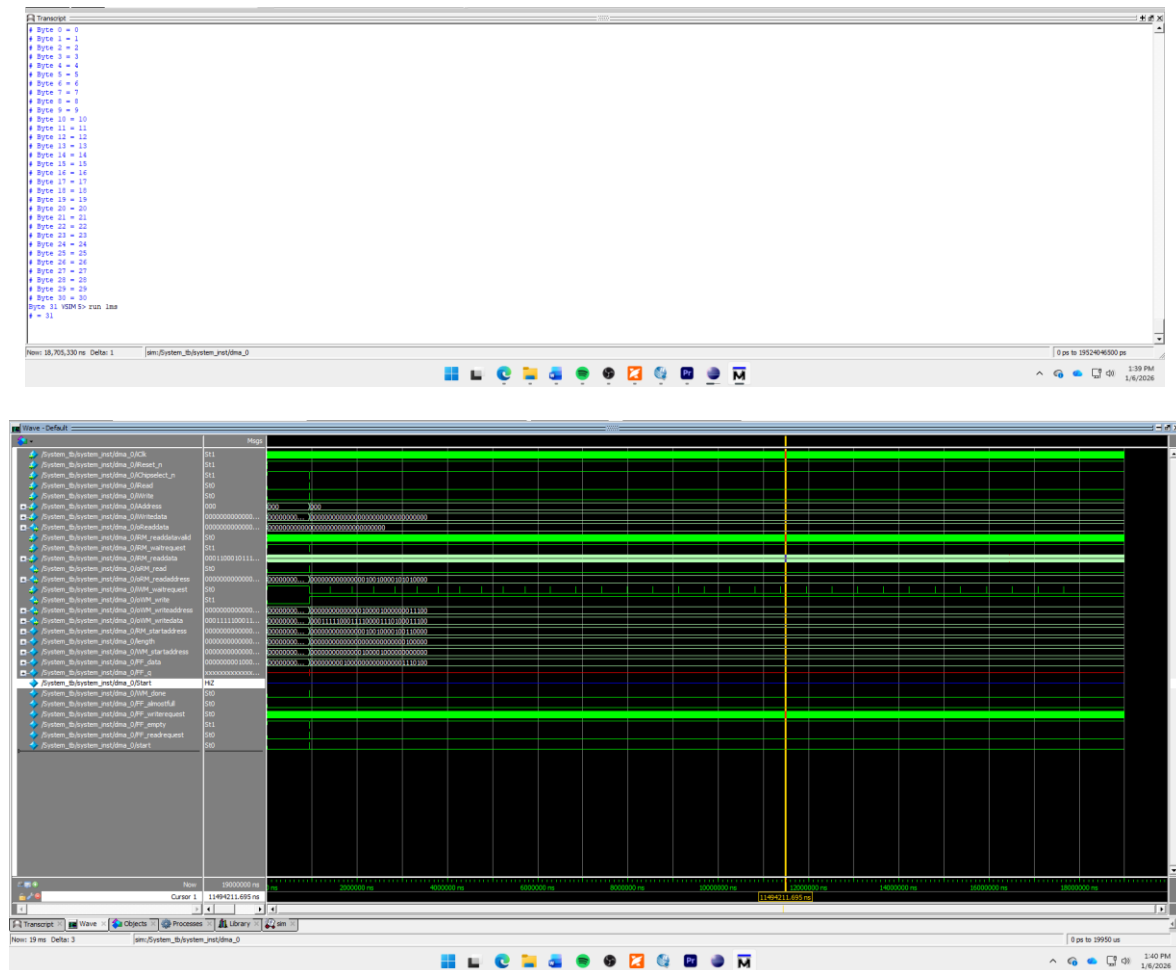
//--tăng địa chỉ đọc lên 1
always @ (posedge iClk or negedge iReset_n) //đổi iReset_n thành bất đồng bộ
begin
    if(~iReset_n)
        pos_read <= 8'b00000000;
    else if(fifo_rd)
        pos_read <= pos_read + 1;
    end

    //-----
    assign compare = pos_write[7] ^ pos_read[7]; //so sánh bit cao của pos_read và pos_write
    assign equal = (pos_write[6:0] - pos_read[6:0]) ? 0:1;
    assign FF_almostfull = compare & equal; //Bao FF_almostfull = 1 khi buffer/2
    assign FF_empty = ~compare & equal; //Bao FF_empty=1 khi pos_read = pos_write
endmodule

```

Bảng 2: Chỉnh sửa file FIFO.v

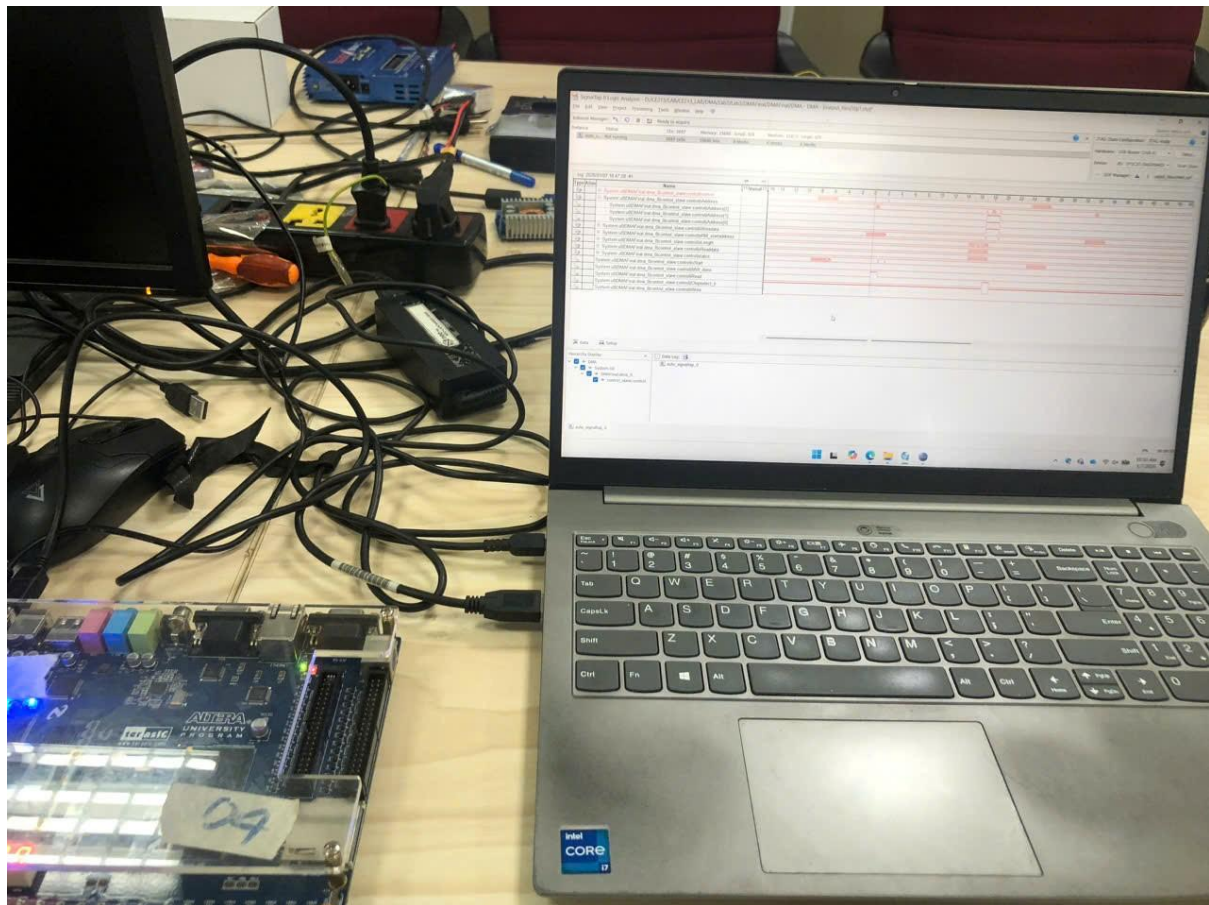
Sau khi đã chỉnh sửa file lại ta làm lại các bước như trong video cho trước thì kết quả cho ra sẽ được như thế này.

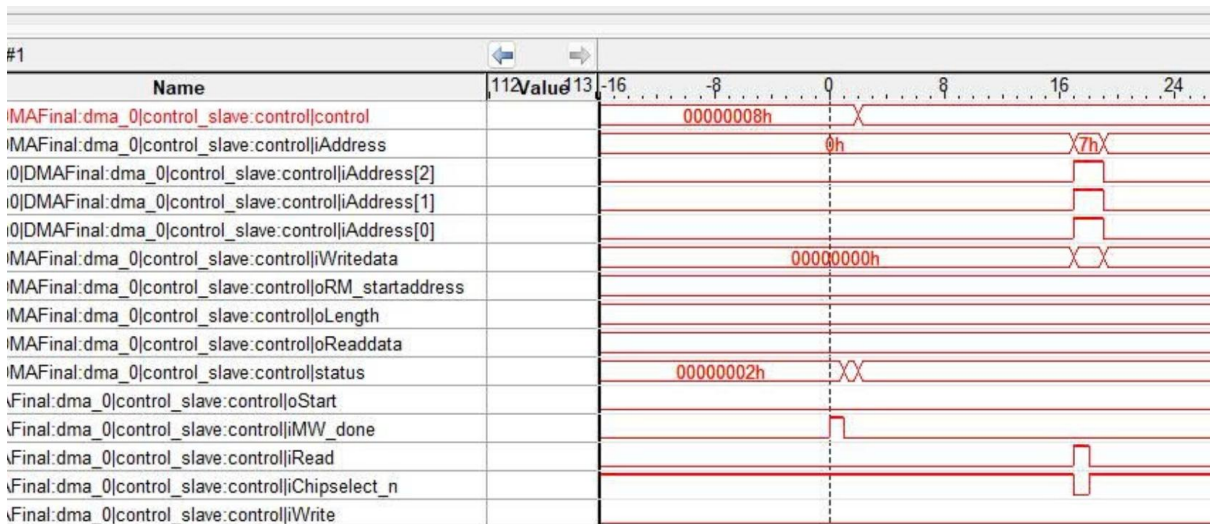
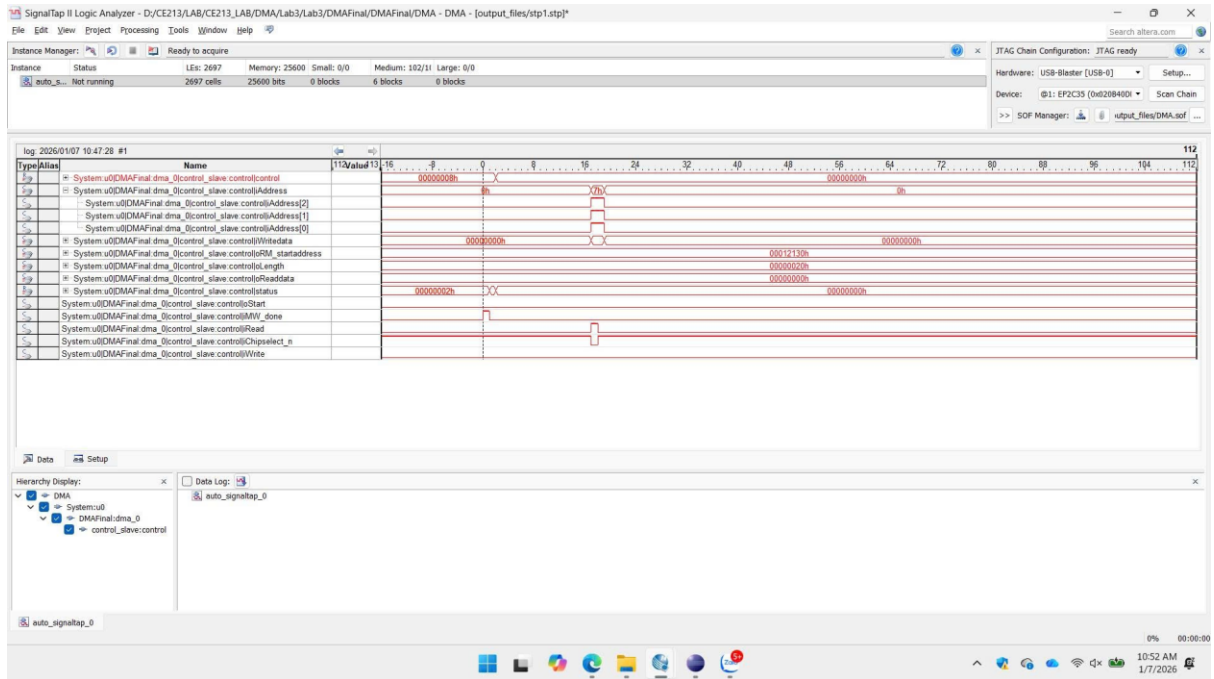


Hình 3: Kết quả đúng khi mô phỏng

CHƯƠNG IV: HIỆN THỰC LÊN KIT DE2

1. Kết quả khi nạp lên KIT DE2





Hình 4: Kết quả cho ra khi nạp lên DE2

Để bắt được tín hiệu DONE như thế này ta cần phải điều chỉnh Trigger của tín hiệu đó như sau

trigger: 2026/01/07 14:25:25 #0			Lock mode: Allow all changes		
Node			Data Enable	Trigger Enable	Trigger Conditions
Type	Alias	Name	200	200	1 Basic AND
		System:u0 DMAFinal:dma_0 control_slave:control control			XXXXXXXXh
		System:u0 DMAFinal:dma_0 control_slave:control iAddress			xh
		System:u0 DMAFinal:dma_0 control_slave:control iAddress[2]			
		System:u0 DMAFinal:dma_0 control_slave:control iAddress[1]			
		System:u0 DMAFinal:dma_0 control_slave:control iAddress[0]			
		System:u0 DMAFinal:dma_0 control_slave:control iWritedata			XXXXXXXXh
		System:u0 DMAFinal:dma_0 control_slave:control oRM_startaddress			XXXXXXXXh
		System:u0 DMAFinal:dma_0 control_slave:control oLength			XXXXXXXXh
		System:u0 DMAFinal:dma_0 control_slave:control oReaddata			XXXXXXXXh
		System:u0 DMAFinal:dma_0 control_slave:control iStatus			XXXXXXXXh
		System:u0 DMAFinal:dma_0 control_slave:control oStart			
		System:u0 DMAFinal:dma_0 control_slave:control iMW_done			
		System:u0 DMAFinal:dma_0 control_slave:control iRead			
		System:u0 DMAFinal:dma_0 control_slave:control iChipselect_n			
		System:u0 DMAFinal:dma_0 control_slave:control iWrite			

Hình 5: Trigger Conditions của *iMW_done* thành *Rising Edge*

Ta chuyển Trigger Conditions của *iMW_done* từ *x* thành *Rising Edge* để sẵn sàng bắt tín hiệu ngay khi *iMW_done* thỏa mãn điều kiện đó.

Như vậy là ta đã bắt được tín hiệu *iMW_done* ngay khi nó tích cực cạnh lên thành công!

CHƯƠNG III. KẾT LUẬN.

Bài thực hành nhìn chung yêu cầu sinh viên tìm hiểu về kiến trúc DMA, các thành phần cần thiết, cách nó hoạt động trên FPGA, đồng thời kiểm chứng cách hoạt động của nó.